

Diviser pour Régner

Méthode Générale

On considère un problème P à résoudre sur un nombre n de données.

Méthode de résolution

Partager le problème en k sous problème $1 \leq k \leq n$

Calculer les solutions des k sous-problèmes $\Rightarrow$ k solutions

Trouver une méthode pour combiner les k solutions afin d'engendrer la solution du problème initial

Les sous-problèmes sont traités indépendamment

On se donne une fonction T qui calcule le coût relative à la résolution de P.

Schéma Général

On considère un tableau A [1...n] contenant les n données de départ.

Soient p et q les bornes sur lesquelles on travaille et m tel que $p \leq m \leq q$

La procédure Diviser pour Régner pour une division du problème initial en deux sous problèmes peut s'écrire comme suit :

```
DivRegn (p,q)
Si Int (p,q) alors Res (p,q)
Sinon
/* m position dans le tableau où on va appliquer la division*/
m=diviser (p,q)
R1=DivRegn (p,m)
R2=DivRegn (p,q)
R=Combiner (R1,R2)
Finsi
```

Int(p,q): vrai si l'intervalle d'étude [p,q] permet de trouver facilement une solution

Diviser(p,q) : entier la position de la division des données (m)

Res(p,q) : fonction qui calcule le résultat

Combiner(R1,R2) : combiner les solution R1 et R2 pour trouver la solution du problème initial (R)

Complexité du Problème

On définit par $T(n)$: Temps d'exécution pour n données

Pour une division de l'intervalle de travail en 2 $n \Rightarrow n/2$ $m=(q-p+1)/2$

$$T(n) = \begin{cases} r(n) & \text{si } n \text{ est petit (calcul du résultat)} \\ 2 T\left(\frac{n}{2}\right) + f(m) & (f(m) \text{ combiner et diviser}) \end{cases}$$

Exemple : Tour de Hanoï

On dispose de 3 piquets A,B,C et n anneaux $a_1, a_2, \dots, a_n$ de rayons $r_1, r_2, \dots, r_n$ tels que $r_1 \leq r_2 \leq \dots \leq r_n$ placés initialement sur le piquet A de bas vers le haut on a : $a_n, a_{n-1}, \dots, a_2, a_1$.

Problème posé :

Comment déplacer les anneaux du piquet initial A vers un autre (B) tout en respectant les contraintes suivantes ?

- Un anneau a_i ne doit pas être placé sur une pile d'anneaux (ou un anneau) de rayon(s) inférieur(s) à a_i
- Un seul anneau est déplacé à la fois
- L'anneau à déplacer est celui se trouvant au sommet de la tour

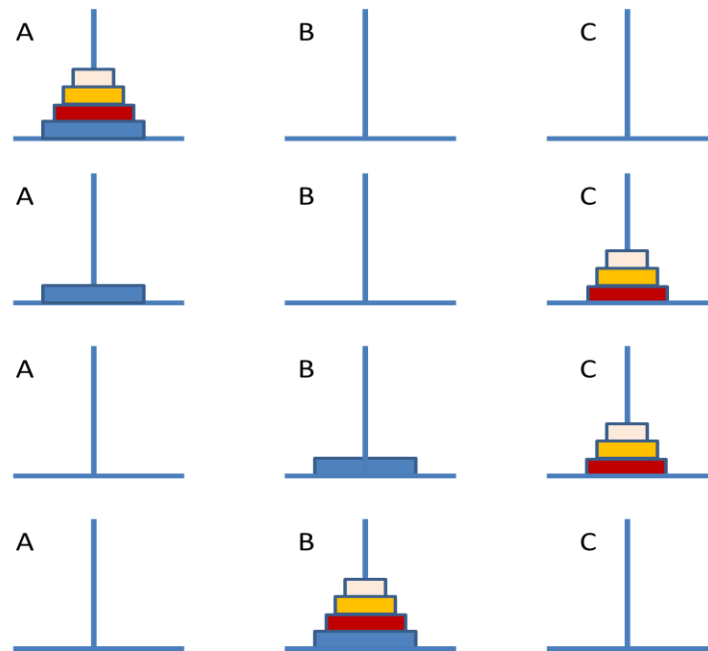
Idée de résolution

L'idée de résolution du problème de taille n revient à résoudre deux sous problèmes de taille $n-1$ et ce, de la manière suivante :

Déplacer les $(n-1)$ plus petits anneaux de A vers C (en utilisant B)

Déplacer l'anneau n vers B

Déplacer les $(n-1)$ anneaux de C vers B

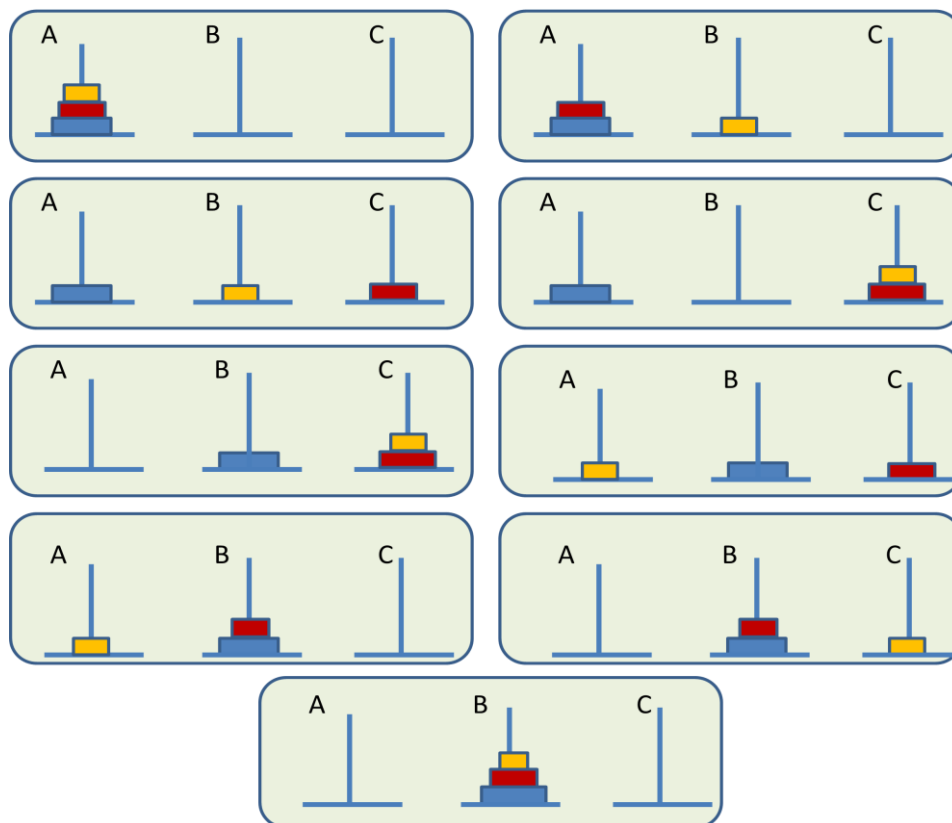


Principe de résolution du problème de la tour de hanoï

Complexité : opération fondamentale : déplacement

$$T(n)=2T(n-1)+1 \quad T(0)=0 \Rightarrow T(n)=2^n-1 \quad T(n) \text{ en } O(2^n)$$

Exemple pour n=3



Résolution de la Tour de Hanoï pour n=3

Exemple : Recherche du plus petit et du plus grand élément d'un ensemble S de taille $n=2^k$

Version itérative

Algorithme MinMax Min <- un élément quelconque y de S Max <- y Pour chaque élément x de $S \setminus \{y\}$ faire si $x < \text{Min}$ alors Min <- x Fin si si $x > \text{Max}$ alors Max <- x Fin si Fin pour	Algorithme MinMax2 Min <- T[1] Max <- T[1] Pour i de 2 à n faire si $T[i] < \text{Min}$ alors Min <- T[i] Fin si si $T[i] > \text{Max}$ alors Max <- T[i] Fin si Fin pour
---	--

Complexité temporelle : opération fondamentale : comparaison

$2(n-1)$ comparaisons

Version récursive

```

Algorithme
Fonction MinMaxR ( ensemble S)
Si card (S)=2 et  $S=\{x_1, x_2\}$  alors
    Si  $(x_1 < x_2)$  alors
        retourne  $(x_1, x_2)$ 
    Sinon
        retourne  $(x_2, x_1)$ 
Finsi
Sinon
    Diviser S en S1 et S2 de même taille
     $(a_1, b_1) = \text{MinMaxR}(S_1)$ 
     $(a_2, b_2) = \text{MinMaxR}(S_2)$ 
    Retourne  $(\min(a_1, a_2), \max(b_1, b_2))$ 
Finsi
    
```

Complexité temporelle : opération fondamentale : comparaison

Nombre de comparaisons en fonction de la taille n de S

$$T(n) = \begin{cases} 1 & \text{si } n=2 \\ 2T(n/2)+2 & \text{si } n>2 \end{cases}$$

Cette équation a pour solution la fonction $T(n) = \frac{3}{2}n - 2$

25% meilleure que la version itérative

Théorème

Soit T une fonction croissante de $\mathbb{R} \rightarrow \mathbb{R}$ a, b réels positifs et $c > 1$ telle que

$$T(1) = b$$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = a T\left(\frac{n}{c}\right) + bn$$

$$\text{Alors } T(n) \text{ est en } \begin{cases} O(n) & \text{si } a < c \\ O(n \log_c(n)) & \text{si } a = c \\ O(n^{\log_c(a)}) & \text{si } a > c \end{cases}$$

Lemme

Soit la suite U définie par :

$$\begin{cases} U_0 = \mu_0 \\ U_m = \lambda_m U_{m-1} + \mu_m \end{cases}$$

$$\text{Alors } U_m = \sum_{i=0}^m \mu_i \lambda_{i+1} \dots \lambda_m$$

Preuve du lemme

$$\begin{cases} U_m = \lambda_m U_{m-1} + \mu_m \\ U_m = \lambda_m (\lambda_{m-1} U_{m-2} + \mu_{m-1}) + \mu_m \\ \quad = \lambda_m \lambda_{m-1} U_{m-2} + \lambda_m \mu_{m-1} + \mu_m \\ \quad \quad \vdots \\ \quad \quad \quad \vdots \\ \lambda_m \lambda_{m-1} \dots \lambda_1 \mu_0 + \lambda_m \lambda_{m-1} \dots \lambda_2 \mu_1 + \lambda_m \mu_{m-1} + \mu_m \\ \quad \quad = \sum_{i=0}^m \mu_i \lambda_{i+1} \dots \lambda_m \end{cases}$$

Preuve du théorème

On pose $n = c^k$

$T(n) = T(c^k)$ qu'on note T_k

$$T(1) = T(c^0) = T_0$$

$$T(n) = aT(n/c) + bn \Rightarrow T(c^k) = aT(c^k/c) + bc^k$$

$$T_0 = b$$

$$\left\{ \begin{array}{l} T_k = aT(c^{k-1}) + bc^k \\ \text{(lemme)} = \sum_{i=0}^k bc^i + a^{k-i} \text{ avec } bc^i = \mu_i \text{ et } \lambda_i = a \\ = ba^k + bca^{k-1} + bc^2a^{k-2} + \dots + bc^k \\ = bc^k \sum_{i=0}^k \frac{a^{k-i}}{c^{k-i}} \\ = bc^k \sum_{i=0}^k \frac{a^i}{c^i} \\ = bc^k \sum_{i=0}^k \left(\frac{a}{c}\right)^i \end{array} \right.$$

$$n = c^k \Rightarrow k = \log_c(n)$$

$$T(n) = bn \sum_{i=0}^{\log_c(n)} \left(\frac{a}{c}\right)^i$$

Si $a < c$ la série de terme général $\left(\frac{a}{c}\right)^i$ converge vers 1

$$\left(\frac{a}{c}\right)^0 + \left(\frac{a}{c}\right)^1 + \dots + \left(\frac{a}{c}\right)^k = 1 + \ll + \dots + \ll \cong 1$$

$$T(n) = bn \Rightarrow T(n) : O(n)$$

Si $a = c$

$$T(n) = bn[1 + 1 \dots + 1] = bn(k + 1) \Rightarrow T(n) = bn(\log_c(n) + 1) \Rightarrow T(n) : O(n \log_c(n))$$

Si $a > c$

$$\sum_{i=0}^k \left(\frac{a}{c}\right)^i = 1 * \frac{1 - \left(\frac{a}{c}\right)^{k+1}}{1 - \left(\frac{a}{c}\right)} = \frac{\left(\frac{a}{c}\right)^{k+1} - 1}{\left(\frac{a}{c}\right) - 1} = \left(\frac{a}{c}\right)^k \frac{\left(\frac{a}{c}\right) - \frac{1}{\left(\frac{a}{c}\right)^k}}{\left(\frac{a}{c}\right) - 1} \cong \left(\frac{a}{c}\right)^k (k \rightarrow \infty)$$

$$\sum_{i=0}^k \left(\frac{a}{c}\right)^i = \left(\frac{a}{c}\right)^k$$

$$\left\{ \begin{array}{l} T(n) = bn \left(\frac{a}{c}\right)^k = bc^k \frac{a^k}{c^k} = ba^k \\ = ba^{\log_c(n)} = b \exp^{\log(a^{\log_c(n)})} \\ = b \exp^{(\log_c(n) * \log(a))} = b \exp\left(\frac{\log(n)}{\log(c)} * \log(a)\right) \\ = b \exp\left(\log(n) * \frac{\log(a)}{\log(c)}\right) = b \exp(\log(n) * \log_c(a)) \\ b \exp^{\log(n^{\log_c(a)})} = bn^{\log_c(a)} \\ \Rightarrow T(n) = bn^{\log_c(a)} \end{array} \right.$$

$$T(n) : O(n^{\log_c(a)})$$

Exemples

$$T(0) = b_0$$

$$T(n) = a T\left(\frac{n}{c}\right) + bn$$

$$\mathbf{a-} \quad T(n) = T\left(\frac{n}{2}\right) + n \quad a=1 ; c=2 ; a < c \quad T(n) : O(n)$$

$$\mathbf{b-} \quad T(n) = 2T\left(\frac{n}{2}\right) + 4n \quad a=2 ; c=2 ; a = c \quad T(n) : O(n \log_2(n))$$

$$\mathbf{c-} \quad T(n) = 4T\left(\frac{n}{3}\right) + n \quad a=4 ; c=3 ; a > c \quad T(n) : O(n^{\log_3(4)}) = O(n^{2 \log_3(2)})$$

T(n) ? par calcul

$$\mathbf{a-} \quad T(n) = T\left(\frac{n}{2}\right) + n \quad \mathbf{a'-} \quad T(n) = T\left(\frac{n}{2}\right) + 1$$

$$\mathbf{b-} \quad T(n) = 2T\left(\frac{n}{2}\right) + 4n \quad \mathbf{b'-} \quad T(n) = 2T\left(\frac{n}{2}\right) + 4$$

$$\mathbf{c-} \quad T(n) = 4T\left(\frac{n}{3}\right) + n \quad \mathbf{c'-} \quad T(n) = 4T\left(\frac{n}{3}\right) + 1$$

Théorème

Soit T une fonction croissante de $\mathbb{R} \rightarrow \mathbb{R}$ a, b réels positifs et $c > 1$ telle que

$$T(1) = b$$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = a T\left(\frac{n}{c}\right) + bn^k$$

$$\text{Alors} \quad T(n) \text{ est en } \begin{cases} O(n^k) & \text{si } a < c^k \\ O(n^k \log_c(n)) & \text{si } a = c^k \\ O(n^{\log_c(a)}) & \text{si } a > c^k \end{cases}$$

Exemple1 : Multiplication de deux entiers sur n bits

Soient l'écriture binaire de deux entiers X et Y sur n bits

X :

--	--	--	--	--	--	--	--

Y :

--	--	--	--	--	--	--	--

X et Y peuvent être exprimés également comme des entiers définis sur n/2 bits

X :

A	B
---	---

→ X : $A 2^{n/2} + B$

exple 8 sur 4 bits = 1000

2^3	2^2	2^1	2^0
1	0	0	0

$$8 = 2^3 = 2 \cdot 2^2 + 0$$

Y :

C	D
---	---

→ Y : $C 2^{n/2} + D$

Multiplication de X par Y

Méthode 1

$$X * Y = (A 2^{n/2} + B) * (C 2^{n/2} + D)$$

$$= A C 2^n + (AD + BC) 2^{n/2} + BD$$

→ 4 multiplications d'entiers de taille $n/2$ AC, AD, BC et BD

3 additions d'au plus 2^n

2 multiplications par 2^n et $2^{n/2}$

L'expression de $T(n)$ associée à ce traitement

$$T(1) = 1$$

$$T(n) = 4T\left(\frac{n}{2}\right) + cte \cdot n \quad / \quad cte \cdot n \text{ correspond au nombre d'additions et multiplications } 2^n \text{ et } 2^{n/2}$$

$$T(n) = a T\left(\frac{n}{c}\right) + bn$$

$$a > c \quad (4 > 2) \quad \rightarrow \quad T(n) \text{ en } O(n^{\log_c(a)}) \quad T(n) \text{ en } O(n^{\log_2(4)})$$

$$T(n) \text{ en } O(n^{2 \log_2(2)}) \quad T(n) \text{ en } O(n^2)$$

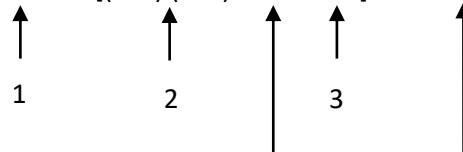
Méthode 2

$$X * Y = (A 2^{n/2} + B) * (C 2^{n/2} + D)$$

$$= A C 2^n + (AD + BC) 2^{n/2} + BD$$

Une optimisation est de considérer :

$$A D + B C = (A-B) (D-C) + A C + B D \quad \rightarrow \quad X * Y = A C 2^n + [(A-B) (D-C) + A C + B D] 2^{n/2} + BD$$



→ 3 multiplications de $n/2$ bits

Déjà calculé

$$T(1) = 1$$

$$T(n) = 3T\left(\frac{n}{2}\right) + cte \cdot n \quad / \text{cte } n \text{ correspond au nombre d'additions et multiplications } 2^n \text{ et } 2^{n/2}$$

$$T(n) \text{ en } O(n^{\log_2(3)})$$



$$T(n) \text{ en } O(n^{1.58}) < O(n^2)$$

Exemple2 : Multiplication de deux matrices carrées

On considère deux matrices A (nxn) et B(nxn) et la matrice résultat C=A*B qui est aussi C(nxn)

Méthode 1 :classique ; version IJK

Pour la version IJK (classique), le calcul de C est déterminé par :

$$C[i,j] = \sum_{k=1}^n A[i,k] * B[k,j] \quad 1 \leq i \leq n \quad 1 \leq j \leq n$$

Algorithme version IJK

```

Pour i de 1 à n faire
  Pour j de 1 à n faire
    C[i,j]=0
    Pour k de 1 à n faire
      C[i,j]=C[i,j]+A[i,k]*B[k,j]
    Fin pour
  Fin pour
Fin pour

```

Complexité temporelle : opération fondamentale : multiplication et l'addition

$$T(n) \text{ en } O(n^3)$$

Les autres versions (IKJ, JIK, JKI, KIJ, KJI) diffèrent de la manière de calculer les éléments de C mais la complexité est en $O(n^3)$

Méthode 2 :Méthode de Strassen

En 1968 Strassen a proposé une autre méthode dont le but de réduire la complexité.

Pour des matrices d'ordre 2 on a :

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix} = \begin{bmatrix} x1 + x2 - x4 + x6 & x4 + x5 \\ x6 + x7 & x2 - x3 + x5 - x7 \end{bmatrix}$$

Avec

$x1 = (b - d)(g + h)$	$x4 = (a + b)h$
-----------------------	-----------------

$x2 = (a + d)(e + h)$	$x5 = a(f - h)$
$x3 = (a - c)(e + f)$	$x6 = d(g - e)$
	$x7 = (c + d)e$

On a effectué **7 multiplications et 18 additions** au lieu de **8 multiplications et 4 additions**

Sachant que le coût d'une multiplication est nettement supérieure par rapport à celui d'une addition cette méthode serait moins coûteuse.

Dans le cas général, On suppose que la taille des matrices $n=2^k$ pour pouvoir diviser récursivement les matrices en 4 et ce, en s'inspirant du modèle de taille 2, comme suit :

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \text{ avec } A_{ij} \text{ de taille } n/2 \Rightarrow (2^{k-1})$$

L'algorithme de multiplication de deux matrices va utiliser les expressions précédentes et les procédures suivantes :

ProdMat(A,B,n) , AddMat(A,B,n), SubMat(A,B,n)

Algorithme Fonction ProdMat(A,B,n)

```

Si n=1 alors prodMat := A*B
Sinon
    Décomposer_Mat (A)  ⇨    A11 ; A12 ; A21 ; A22
    Décomposer_Mat (B)  ⇨    B11 ; B12 ; B21 ; B22
Finsi
X1=ProdMat(SubMat(A12,A22,n/2),AddMat(B21,B22,n/2) ,n/2);
X2=ProdMat(AddMat(A11,A22,n/2),AddMat(B11,B22,n/2),n/2) ;
:
X7= ProdMat(AddMat(A21,A22,n/2),B11,n/2) ;
C11:=SubMat(AddMat(X1,X2,n/2),SubMat(X4,X6,n/2),n/2);
:
C22:=AddMat(SubMat(X2,X3,n/2),SubMat(X5,X7,n/2),n/2);
Recompose (C)
Fin

```

ProdMat (... ,N) fait 7 appels à ProdMat(...,n/2) et 18 appels à AddMat(...,n/2)

L'addition de deux matrices est en $O(n^2)$ alors que la multiplication est en $O(n^3)$

Complexité méthode 1 :

$$T(1) = b$$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = 8 T\left(\frac{n}{2}\right) + cte n^2$$

Une addition $n/2 * n/2 \Rightarrow n^2/4$ éléments $\Rightarrow \frac{1}{4} n^2 \Rightarrow$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = 8 T\left(\frac{n}{2}\right) + cte n^2$$

$$T(n) \in O(n^{\log_2(8)}) = O(n^3)$$

Complexité méthode 2 :

$$T(1) = b$$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = 7 T\left(\frac{n}{2}\right) + cte n^2$$

Une addition $n/2 * n/2 \Rightarrow n^2/4$ éléments $\Rightarrow \frac{1}{4} n^2 \Rightarrow$ on a 18 additions

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = 7 T\left(\frac{n}{2}\right) + \frac{18}{4} n^2$$

$$n \in \mathbb{N} \quad n > 1 \quad T(n) = 7 T\left(\frac{n}{2}\right) + 18 \left(\frac{n}{2}\right)^2$$

$$T(n) \in O(n^{\log_2(7)}) = O(n^{2,807})$$



$$T(n) \in O(n^{2,807}) < O(n^3)$$